TEZPUR UNIVERSITY

Bachelor of Technology

COMPUTER SCIENCE AND ENGINEERING

Project Report on

GROCERY STORE MANAGEMENT SYSTEM

CO317: PROJECT- I (USING SE PERSPECTIVE)

Submitted By
ANWESHA SAHARIA
Roll No : CSB23042

Under the guidance of
PROF. SANJIB K. DEKA
Professor

6th Semester, 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

TEZPUR UNIVERSITY

TEZPUR UNIVERSITY

Tezpur University, Napaam, Sonitpur,
Assam-784028

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled "**Grocery Store Management System**" submitted by Anwesha Saharia (Roll No. CSB23042) as a part of the Course CO317: Project I (Using Software Engineering Perspective) is a bona fide record of work carried out by her during Spring Semester 2026 under my supervision and guidance.

The work embodied in this project report is an original work carried out by the student and has been submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering of Tezpur University.

To the best of my knowledge and belief, the work presented in this project report has not been submitted, either in part or in full, to any other university or institution for the award of any degree, diploma, or other academic qualification.

Guide	Coordinator	Head
Signature	Signature	Signature .....
Name	Name	Name .....

Name of Examiners: **Signature with Date:**

1. _____ _____

2. _____ _____

3. _____ _____

Acknowledgement

I would like to express my sincere gratitude to my project guide - Prof. Sanjib K. Deka, for his invaluable guidance, encouragement, and continuous support throughout the development of this project titled “Grocery Store Management System.” His expertise, suggestions, and constructive feedback played a significant role in helping me successfully complete this work.

I am also thankful to the Department of Computer Science and Engineering and the institution for providing the necessary facilities, resources, and a conducive learning environment that enabled me to carry out this project effectively.

I would like to extend my heartfelt appreciation to my family for their constant encouragement, patience, and unwavering support throughout the course of this project. Their motivation inspired me to overcome challenges and remain committed to achieving my goals.

This project has provided me with an excellent opportunity to apply the theoretical concepts learned during my academic studies to a practical software development environment. The experience gained during the planning, design, implementation, and testing phases has significantly enhanced my technical knowledge and problem-solving skills.

I am grateful to everyone who directly or indirectly contributed to the successful completion of this project.

Anwesha Saharia
Roll No.: CSB23042
B.Tech, 6th Semester
Department of Computer Science and Engineering

Abstract

The Grocery Store Management System (GSMS) is a web-based application developed to automate and simplify the daily operations of a grocery store. Traditional grocery store management often relies on manual record-keeping, which can lead to errors in inventory tracking, billing, and sales management. The proposed system addresses these challenges by providing a centralized platform for managing products, customers, orders, and inventory efficiently.

The system is developed using React.js for the frontend and Django REST Framework for the backend, following a client-server architecture. It provides separate functionalities for administrators and customers through role-based access control. Administrators can manage products, update stock levels, and view customer orders. Customers can register, browse products, maintain a wishlist, add items to a shopping cart, manage delivery addresses, place orders, and view their order history.

The system automatically calculates billing amounts, updates inventory after successful purchases, and maintains organized records within a centralized database. By reducing manual effort and improving data accuracy, the application enhances operational efficiency and provides a convenient shopping experience for customers.

The Grocery Store Management System demonstrates the practical application of modern web technologies and serves as a scalable foundation for future enhancements such as online payment integration, order tracking, and mobile application support.

Contents

1 Introduction

The rapid growth of information technology has led to the adoption of computerized systems in various business domains to improve efficiency, accuracy, and productivity. Grocery stores traditionally rely on manual methods for managing products, inventory, billing, and customer records. Such manual processes are often time-consuming, prone to errors, and difficult to maintain as the volume of transactions increases.

The Grocery Store Management System is a web-based application developed to automate and streamline the day-to-day operations of a grocery store. The system provides a centralized platform for managing products, inventory, customer purchases, billing, and order records. It offers separate functionalities for administrators and customers through role-based access control, ensuring secure and efficient operation.

The project has been developed using modern web technologies, with React.js used for the frontend, Django REST Framework for the backend, and a relational database for data storage. The system aims to simplify store management, improve customer convenience, and maintain accurate records of transactions and inventory.

1.1 Scope

The scope of the Grocery Store Management System includes the automation of essential grocery store operations for both administrators and customers.

For administrators, the system provides facilities to manage products, update inventory, monitor stock availability, and view customer orders. The administrator can add new products, modify existing product information, delete products, and maintain accurate inventory records.

For customers, the system provides a convenient online shopping experience by allowing them to register, log in, browse available products, add products to a shopping cart or wishlist, manage delivery addresses, place orders, and view their order history.

The system maintains centralized records of customers, products, addresses, shopping carts, wishlists, and orders. It also automatically updates inventory levels when purchases are made and generates billing information during the checkout process.

The current implementation is intended for a single grocery store environment and focuses on core store management functionalities. Advanced features such as online payment gateway integration, order tracking, and mobile application support are considered part of the future scope of the project.

1.2 Purpose of the Project

The primary purpose of the Grocery Store Management System is to provide a simple, efficient, and reliable solution for managing grocery store operations through automation. The system aims to replace manual record-keeping methods with a centralized digital platform that improves accuracy and reduces administrative effort.

The project is designed to assist store administrators in managing products, inventory, and customer orders while providing customers with an easy-to-use interface for browsing products, maintaining wishlists, managing shopping carts, and placing orders.

The system also aims to improve inventory control by automatically updating stock quantities after purchases, maintain organized records of store activities, and provide a seamless shopping experience for customers. By automating routine tasks, the system helps improve operational efficiency, minimize human errors, and support effective decision-making.

1.3 Overview of the Report

This report presents the analysis, design, development, implementation, and evaluation of the Grocery Store Management System.

The report begins with an introduction to the project, followed by the problem statement, system requirements, objectives, and feasibility analysis. It then describes the system architecture, subsystem design, database structure, and implementation details.

Subsequent sections discuss the technologies used, high-level features implemented, limitations, and future enhancements. Finally, the report concludes with a summary of the project's achievements and references used during development.

The report provides a comprehensive overview of the development process and demonstrates how the Grocery Store Management System fulfills its intended objectives of improving store management and enhancing customer convenience.

2 Problem Statement

In order to simplify and streamline the activities of a grocery store, an automated **Grocery Store Management System** is needed. The system should allow:

1. The store administrator to:
    - manage products
    - update inventory
    - generate bills for customers
    - track sales
    - maintain organized records in a centralized database
    - Generate periodic reports
2. A customer to:
    - Choose items from the store item list
    - Add items to their shopping cart
    - Make payment
    - View their shopping history
    - Maintain wishlist

By implementing this system, the store can improve accuracy, reduce manual work, and increase overall efficiency in managing grocery store operations.

Motivation

Small and medium grocery stores often manage their daily operations manually using notebooks or basic spreadsheets. This manual system makes it difficult to efficiently track product inventory, sales transactions, and customer purchases. Store owners frequently face problems such as inaccurate stock records, difficulty in generating bills, time-consuming calculations, and lack of proper sales reports.

Without a computerized system, it becomes challenging to maintain updated information about available products, prices, and stock levels. Manual record-keeping can also lead to human errors, data loss, and inefficiency in managing the store's operations.

3 Software Requirements Specification (SRS)

3.1 Functional Requirements

Admin Functional Requirements:

- The system shall allow the admin to add new products to the system.
- The system shall allow the admin to update product details (name, price, quantity, etc.).
- The system shall allow the admin to delete products from the system.
- The system shall allow the admin to view the list of all products.
- The system shall allow the admin to manage inventory (increase/decrease stock).
- The system shall maintain a centralized database of all records.

Customer Functional Requirements:

- Allow customers to view the list of available products
- Display product details (name, price, availability)
- Allow customers to add items to their wishlist
- Allow customers to add items to their shopping cart
- Allow customers to remove items from the cart
- Allow customers to update the quantity of items in the cart
- Allow customers to proceed to checkout
- Allow customers to make payments

System Functional Requirements

- Customer registration
- The system shall authenticate admin and customer login
- The system shall ensure data consistency in inventory and sales records

3.2 Non-Functional Requirements

Performance

- Quick response to user requests and actions
- Handle basic operations like browsing products and billing smoothly

Security

- Provide login authentication for the admin and customers
- Restrict unauthorized access

Usability

- Simple and easy-to-use interface.
- Understandable for users with basic computer knowledge.

Reliability

- The system shall perform operations like billing and cart management correctly.
- The system shall minimize errors during usage.

Maintainability

The system shall be easy to update and modify.

Availability

The system shall be available whenever required by the store.

3.3 System Requirements

- Database: SQLite
- Frontend: React
- Backend: Django
- Programming Languages: Python, JavaScript

4 Data Flow Diagrams (DFD)

4.1 Level 0 DFD

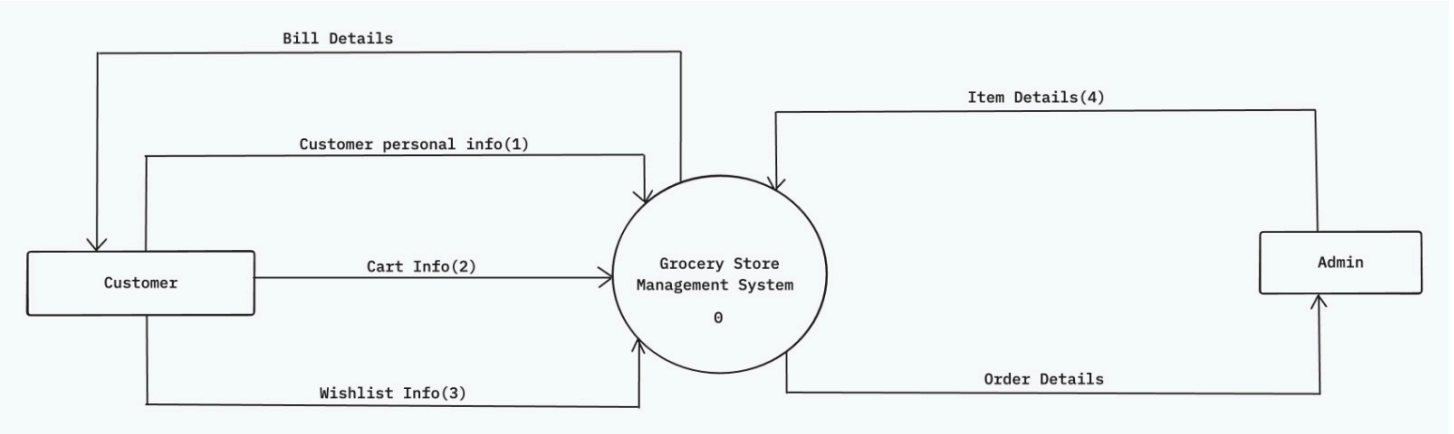


Figure 1: Level 0 Data Flow Diagram

4.2 Level 1

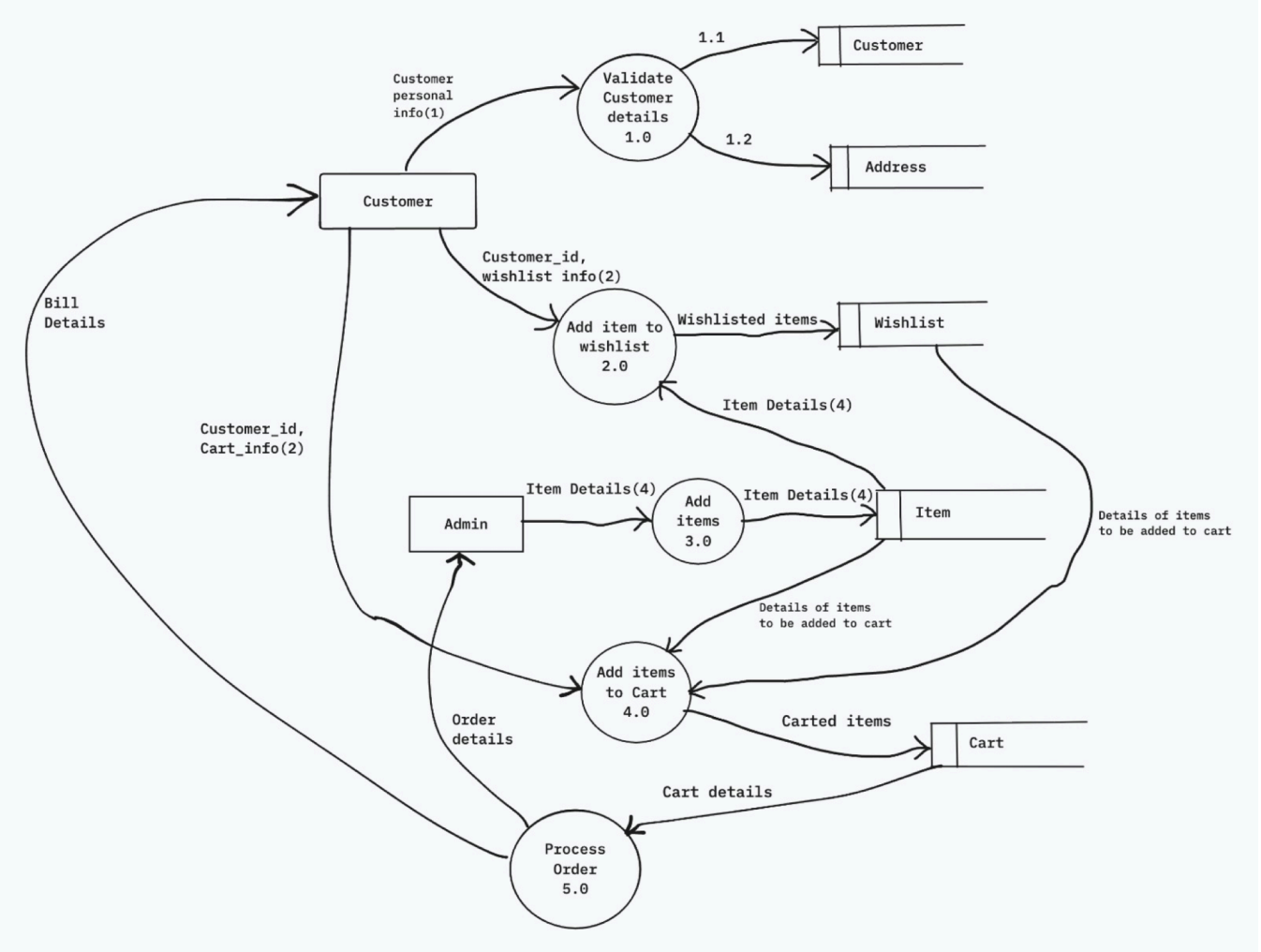
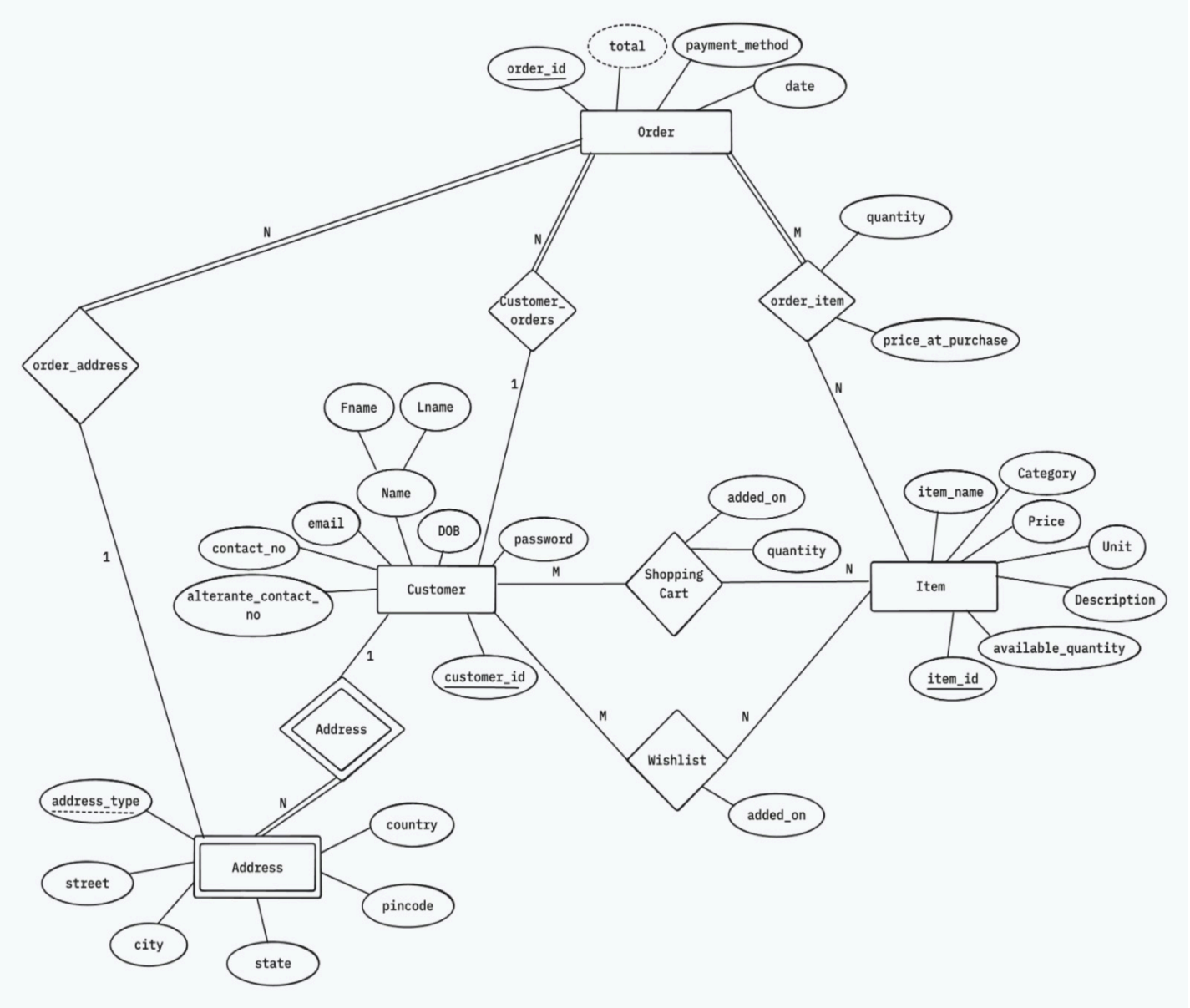
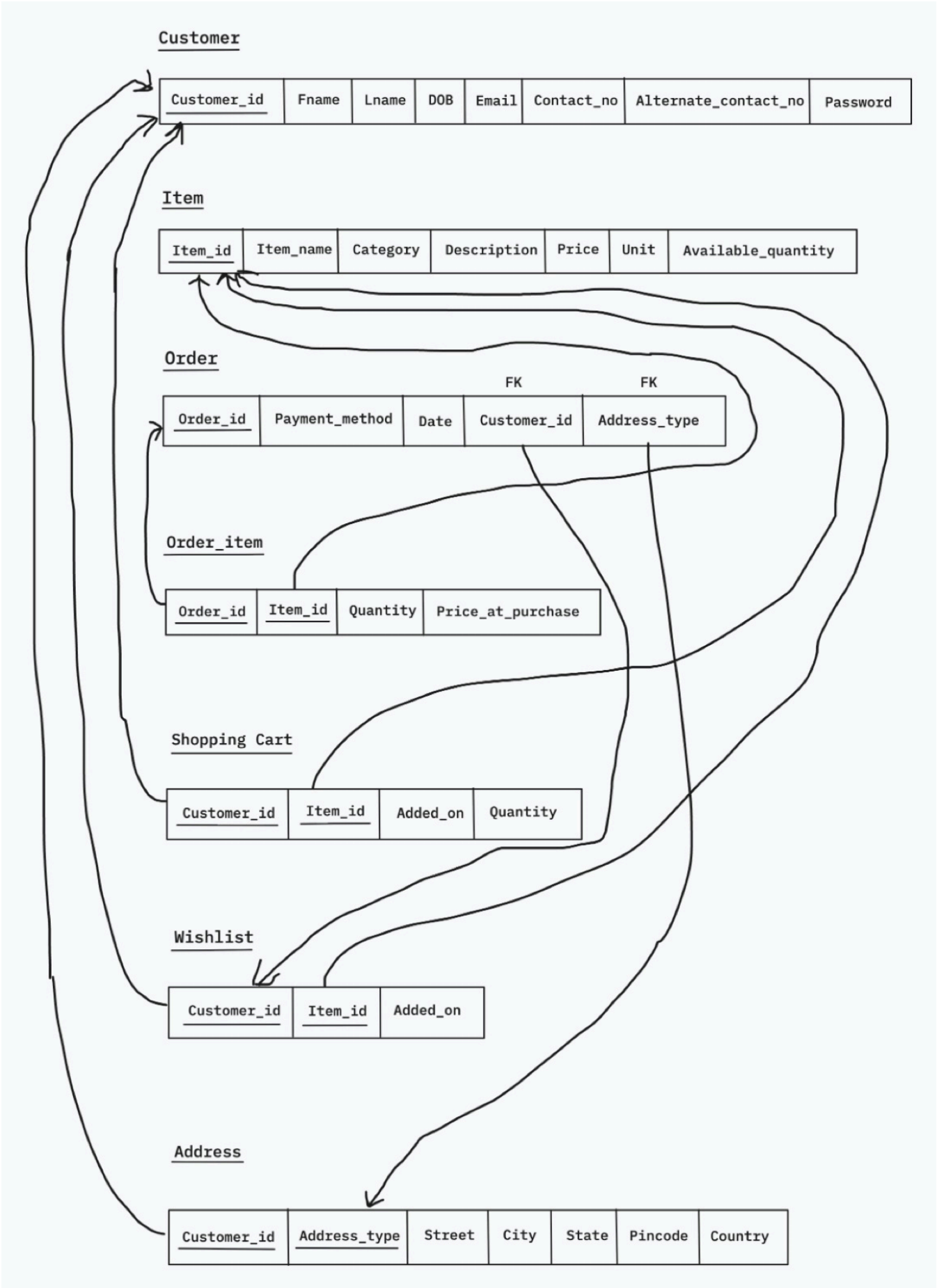


Figure 2: Level 1 Data Flow Diagram

5 Entity Relationship Model (ER Model)



6 Relational Model



7 Objectives

The primary objective of the Grocery Store Management System (GSMS) is to develop a web-based application that automates and simplifies the day-to-day operations of a grocery store. The system is designed to provide an efficient, reliable, and user-friendly platform for managing products, inventory, customer purchases, and sales records.

The specific objectives of the project are:

1. **To automate grocery store operations** by replacing manual record-keeping and billing processes with a centralized digital system.
2. **To facilitate product and inventory management** by enabling the administrator to add, update, delete, and monitor products along with their stock availability.
3. **To provide an efficient shopping experience for customers** by allowing them to browse products, add items to their cart or wishlist, manage delivery addresses, and place orders conveniently.
4. **To generate and maintain accurate sales and order records**, ensuring better tracking of transactions and reducing the possibility of human errors.
5. **To provide secure access** through authentication mechanisms that distinguish between administrator and customer functionalities.
6. **To maintain organized and centralized data storage** for products, customers, orders, addresses, and purchase history, enabling easy retrieval and management of information.

8 Feasibility Study

A feasibility study was conducted to determine whether the proposed Grocery Store Management System is practical and beneficial. The study considered technical, economic, and operational aspects of the system.

1. **Technical Feasibility**

The proposed system is technically feasible because the required technologies, tools, and resources are readily available.

- The system is developed using React.js for the frontend, Django REST Framework for the backend, and SQLite for database management.
- The development tools required, such as Visual Studio Code, Node.js, and Python, are freely available and widely used.
- The hardware requirements are minimal and can be fulfilled using a standard personal computer.
- The technologies used are reliable, scalable, and suitable for implementing features such as product management, shopping cart functionality, billing, order management, and inventory tracking.

Therefore, the project is technically feasible.

2. **Economic Feasibility**

The proposed Grocery Store Management System is economically feasible because the development and operational costs are relatively low compared to the benefits obtained.

- Most of the software and development tools used are open-source and free of cost.
- The system reduces manual paperwork and administrative effort, resulting in lower operational expenses.
- Automated billing and inventory management help minimize human errors and financial losses.
- The centralized database reduces the cost of maintaining physical records.
- Improved efficiency and faster operations can increase customer satisfaction and productivity.

Since the expected benefits outweigh the implementation and maintenance costs, the project is economically feasible.

3. **Operational Feasibility**

The proposed system is operationally feasible because it is designed to be simple, user-friendly, and easy to operate. The interface is intuitive and requires only basic computer knowledge to use. Minimal training is required for users to operate the system effectively.

Therefore, the system can be successfully adopted and used in a real grocery store environment.

9 System Architecture

The Grocery Store Management System follows a three-tier client-server architecture consisting of the Presentation Layer, Application Layer, and Data Layer. This architecture separates the user interface and data storage, making the system modular, maintainable, and scalable.

Architecture Components

1. Presentation Layer (Frontend)
    - Developed using React.js and Tailwind CSS.
    - Provides user interfaces for administrators and customers.
    - Handles user interactions such as product browsing, cart management, billing, and order viewing.
2. Application Layer (Backend)
    - Developed using Django REST Framework.
    - Processes client requests and implements business logic.
    - Manages authentication, inventory updates, cart operations, wishlist operations, address management, and order processing.
3. Data Layer (Database)
    - Stores information related to customers, products, addresses, carts, wishlists, and orders.
    - Ensures data persistence and integrity.

The frontend communicates with the backend through REST APIs, and the backend interacts with the database to retrieve and store information.

9.1 Subsystems and Components

The system is divided into the following subsystems:

1. User Management Subsystem: Responsible for customer registration, login, and authentication.

Components:

- Registration Module
- Login Module
- Role-Based Access Control

2. Product Management Subsystem: Allows administrators to manage grocery products and inventory.

Components:

- Add Product
- Update Product
- Delete Product
- View Products

3. Shopping Cart Subsystem: Allows customers to manage products selected for purchase.

Components:

- Add to Cart
- Increase/Decrease Quantity
- Remove from Cart
- Cart Total Calculation

4. Wishlist Subsystem: Allows customers to save products for future purchase.

Components:

- Add to Wishlist
- Remove from Wishlist
- View Wishlist

5. Address Management Subsystem: Manages customer delivery addresses.

Components:

- Add Address
- View Addresses
- Select Delivery Address

6. Order Management Subsystem: Handles order creation and order history.

Components:

- Place Order
- Order History
- Customer Order Management

7. Billing Subsystem: Generates billing information during checkout.

Components:

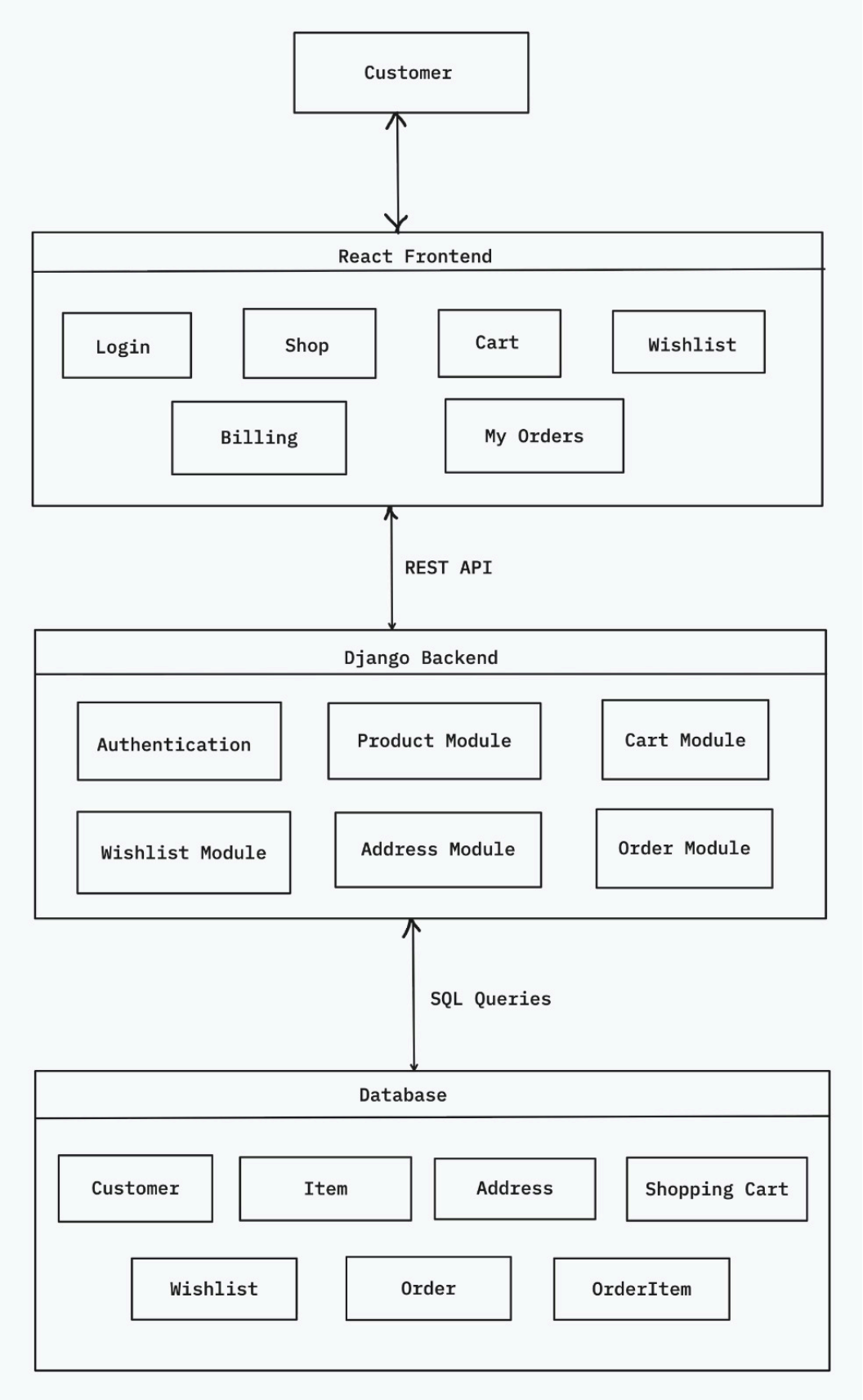
- Bill Calculation
- Payment Method Selection
- Order Confirmation

9.2 Interaction Flow

The interaction flow of the Grocery Store Management System is as follows:

1. A user registers and logs into the system.
2. The customer browses available products from the shop page.
3. Products can be added to the shopping cart or wishlist.
4. The customer reviews cart contents and proceeds to billing.
5. The customer selects an existing delivery address or adds a new address.
6. The customer selects a payment method and places the order.
7. The backend validates stock availability and creates an order.
8. Order details are stored in the database.
9. Product inventory is automatically updated based on purchased quantities.

10. Cart items are cleared after successful order placement.
11. Customers can view their order history through the My Orders page.
12. Administrators can manage products, monitor inventory, and view customer orders through the admin interface.



10 Code Structure

The Grocery Store Management System follows a client-server architecture consisting of a React frontend and a Django REST Framework backend. The code is organized into separate modules to ensure maintainability, scalability, and ease of development.

Frontend Structure (React)

The frontend is responsible for user interaction and presentation. It is organized into the following components:

Reusable UI elements such as:

- Layout
- Navigation Bar
- Product Cards
- Forms

Pages: Individual application screens including:

- Login
- Registration
- Dashboard
- Shop
- Cart
- Wishlist
- Billing
- My Orders
- Products Management

Context API: Used for global state management:

- CartContext
- WishlistContent

Services: Contains API communication logic using Axios to interact with the backend.

Routing: React Router is used for navigation between pages and role-based access control.

Backend Structure (Django REST Framework)

The backend manages business logic, data processing, and database operations.

Models: Database entities used in the system:

- Customer
- Item
- Address
- ShoppingCart
- Wishlist
- Order
- OrderItem

Serializers: Convert model data into JSON format and validate incoming requests.

Views: Contain API endpoints for:

- Authentication
- Product Management
- Cart Operations
- Wishlist Operations
- Address Management
- Order Processing

URLs: Define routes that connect frontend requests to backend views.

Database: Stores all application data including products, customers, addresses, carts, wishlists, and orders.

Working Flow

1. The user interacts with the React frontend.
2. Frontend sends API requests to Django REST Framework.
3. Backend processes requests and interacts with the database.
4. Data is serialized and returned as JSON responses.
5. Frontend updates the user interface accordingly.

11 High-Level Features Implemented

SL no.	Feature	Description
1.	User Authentication	Separate login and registration functionality for customers and administrators with role-based access control.
2.	Product Management	Allows administrators to add, update, delete, and view grocery products along with their details.
3.	Customer Registration	Enables customers to create accounts and store personal information for future purchases.
4.	Address Management	Customers can add, view, and select delivery addresses during checkout.
5.	Product Browsing	Customers can view available products categorized by type along with pricing and stock information.
6.	Shopping Cart	Customers can add products to the cart, update quantities, remove items, and view total costs.
7.	Wishlist Management	Customers can save products to a wishlist for future purchases.
8.	Billing System	Calculates total order amount based on cart contents and selected quantities.
9.	Order Placement	Allows customers to place orders by selecting a delivery address and payment method.
10.	Order History	Customers can view previously placed orders and their details.
11.	Customer Order Management	Administrators can view all customer orders along with customer information and delivery details.
12.	Stock Validation	Prevents customers from adding quantities beyond the available inventory.
13.	Low Stock Monitoring	Highlights products with low inventory levels and identifies out-of-stock products.
14.	Centralized Database	Maintains organized records of customers, products, addresses, carts, wishlists, and orders.
15.	Dashboard	Provides an overview of store-related information and system activities.

12 Limitations

1. Limited Payment Integration: The system currently records payment methods but does not integrate with real online payment gateways such as UPI, credit cards, or digital wallets.
2. No Real-Time Notifications: Customers and administrators do not receive email, SMS, or push notifications regarding order confirmations, deliveries, or stock updates.
3. No Product Image Management: Products are displayed without image support, which limits the visual shopping experience for customers.
4. Single Store Operation: The system is designed for managing a single grocery store and does not support multiple branches or locations.
5. No Order Tracking Status: Customers cannot track the progress of their orders through statuses such as Packed, Shipped, or Delivered.
6. Basic Security Mechanisms: The system provides authentication and authorization but does not include advanced security features such as two-factor authentication or account recovery through email verification.
7. Limited Scalability: The current implementation is suitable for small and medium-sized grocery stores and may require optimization for large-scale deployments with very high transaction volumes.

13 Future Work

The current Grocery Store Management System successfully automates essential store operations such as product management, inventory tracking, shopping cart management, billing, and order processing. However, several enhancements can be incorporated in future versions to improve functionality, user experience, and scalability.

1. Online Payment Gateway Integration: Integrate secure payment gateways such as UPI, Razorpay, PayPal, or Stripe to support real-time online transactions.
2. Product Image Management: Allow administrators to upload product images to provide customers with a more interactive and visually appealing shopping experience.
3. Order Status Tracking: Implement order tracking features with statuses such as Pending, Packed, Shipped, Delivered, and Cancelled to improve order transparency.
4. Notification System: Provide email or SMS notifications for order confirmations, delivery updates, stock alerts, and promotional offers.
5. Multi-Store Support: Extend the system to manage multiple store branches with centralized inventory and sales monitoring.
6. Customer Profile Management: Enable customers to update personal information, manage saved addresses, and view account details through a dedicated profile section.
7. Mobile Application Development: Develop Android and iOS mobile applications to provide convenient access to the system from smartphones and tablets.
8. Recommendation System: Implement intelligent product recommendations based on customer purchase history and preferences using data analytics techniques.
9. Enhanced Security Features: Add features such as two-factor authentication, password recovery via email, and improved access control mechanisms.
10. Inventory Forecasting: Use data analysis and machine learning techniques to predict future stock requirements and prevent shortages.

14 Conclusion

The Grocery Store Management System was developed to automate and simplify the daily operations of a grocery store by replacing manual processes with a centralized digital solution. The system successfully provides functionalities for product management, inventory tracking, customer registration, shopping cart management, wishlist maintenance, billing, address management, and order processing.

The implementation of the system helps reduce manual effort, minimize human errors, and improve the accuracy of store operations. Customers can conveniently browse products, manage their purchases, and view their order history, while administrators can efficiently manage products, inventory, and customer orders through a dedicated interface.

The project demonstrates the effective use of modern web development technologies, including React for the frontend, Django REST Framework for the backend, and a relational database for data management. The modular architecture ensures maintainability and allows future enhancements to be incorporated with ease.

Overall, the Grocery Store Management System achieves its primary objectives of improving operational efficiency, maintaining organized records, and providing a user-friendly platform for both administrators and customers. The system serves as a strong foundation for further expansion through features such as online payments, advanced reporting, order tracking, product image management, and intelligent analytics.

15 References

[1] React Documentation. Available at: https://react.dev/

[2] React Router Documentation. Available at: https://reactrouter.com/

[3] Django Documentation. Available at: https://docs.djangoproject.com/

[4] Django REST Framework Documentation. Available at: https://www.django-rest-framework.org/

[5] Axios Documentation. Available at: https://axios-http.com/docs/intro

[6] Tailwind CSS Documentation. Available at: https://tailwindcss.com/docs

[7] MDN Web Docs. Available at: https://developer.mozilla.org/

[8] SQLite Documentation. Available at: https://www.sqlite.org/docs.html

[9] Python Documentation. Available at: https://docs.python.org/

[10] JavaScript Documentation (MDN). Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[11] HTML Living Standard. Available at: https://html.spec.whatwg.org/

[12] CSS Documentation (MDN). Available at: https://developer.mozilla.org/en-US/docs/Web/CSS

[13] Visual Studio Code Documentation. Available at: https://code.visualstudio.com/docs

[14] Node.js Documentation. Available at: https://nodejs.org/docs/latest/api/

[15] Git Documentation. Available at: https://git-scm.com/doc

16 Appendix

16.1 Github Code Link

https://github.com/AnweshaS25/BTechProject

16.2 Sample Commands

Build Commands:

1. Frontend Setup:

   $ npm install

   $ npm run dev

2. Backend Setup

   $ python manage.py makemigrations

   $ python manage.py migrate

   $ python manage.py runserver

3. Create Admin User

   $ python manage.py createsuperuser